

Secure architecture - Filtering

OUTLINES

1 PRESENTATION

MESSAGE FILTERING

FILTER BETWEEN LOCAL NETWORK AND ATTACKERS

- Criteria = @IP, ports, protocols
- Prevent access from the outside
- Protects from attacks relying on emitter @IP authentication
- Example: a packet coming from the outside has a local @ as source @
- No broadcast and multicast connexions

LIMITS

THE PREVIOUS METHOD IS SOMEWHAT TOO MUCH RIGID

- stops local network services
- ⇒ clears more access than necessary to make the firewall efficient

► Example

STATEFUL FIREWALL

FILTER PACKETS + TRACE CONNEXIONS (ADAPTIVE BEHAVIOUR)

TCP

- 1 The firewall establishes an authorized connexion
- 2 Every segment associated with the connexion will be transmitted

UDP, ICMP: NO CONNEXION

- Mandatory filtering
- ⇒ Specific rules for each protocol
 - Some protocols do not work with this approach (2 ports for FTP)

STATEFUL FIREWALL

FILTER PACKETS + TRACE CONNEXIONS (ADAPTIVE BEHAVIOUR)

TCP

- 1 The firewall establishes an authorized connexion
- 2 Every segment associated with the connexion will be transmitted

UDP, ICMP: NO CONNEXION

- Mandatory filtering
- ⇒ Specific rules for each protocol
 - Some protocols do not work with this approach (2 ports for FTP)

ANALYSIS ACCURACY

CHECK FIREWALLS

- Some firewalls trust already established TCP connexions
- + better performances and easier to implement...
- at the expense of security
- Other firewalls rely on precise criteria to determine the contents of TCP segments

CHOICES RELY UPON SECURITY POLICY

- Possibility of deep scanning
- Know the protocols
- Test firewalls

ANALYSIS ACCURACY

CHECK FIREWALLS

- Some firewalls trust already established TCP connexions
- + better performances and easier to implement...
- at the expense of security
- Other firewalls rely on precise criteria to determine the contents of TCP segments

CHOICES RELY UPON SECURITY POLICY

- Possibility of deep scanning
- Know the protocols
- Test firewalls

LIMITS OF STATEFUL FIREWALLS

- A connexion is always either cleared or denied.
- After clearance for first packet, other packets of the same segment are not always checked.
- No protection against server weaknesses

LIMITS OF STATEFUL FIREWALLS

- A connexion is always either cleared or denied.
- After clearance for first packet, other packets of the same segment are not always checked.
- No protection against server weaknesses

LIMITS OF STATEFUL FIREWALLS

- A connexion is always either cleared or denied.
- After clearance for first packet, other packets of the same segment are not always checked.
- No protection against server weaknesses

BUILDING RULES

- Rules for entering $\neq$ rules for exiting
- A packet is validated, rejected or transferred to another rule
- Successive rules form a **chain**
- Chains are contained into **tables**
 - Filtering does not work with every protocol
 - + Control over ports, packet header and classical protocols

BUILDING RULES

- Rules for entering $\neq$ rules for exiting
- A packet is validated, rejected or transferred to another rule
- Successive rules form a **chain**
- Chains are contained into **tables**
 - Filtering does not work with every protocol
 - + Control over ports, packet header and classical protocols

BUILDING RULES

- Rules for entering $\neq$ rules for exiting
- A packet is validated, rejected or transferred to another rule
- Successive rules form a **chain**
- Chains are contained into **tables**
- Filtering does not work with every protocol
- + Control over ports, packet header and classical protocols

BUILDING RULES

- Rules for entering $\neq$ rules for exiting
- A packet is validated, rejected or transferred to another rule
- Successive rules form a **chain**
- Chains are contained into **tables**
 - Filtering does not work with every protocol
 - + Control over ports, packet header and classical protocols

DEFAULT FILTERING

- Accept or reject according to first concordance
- specific rules checked before general rules

DEFAULT POLICY

Anything that is not specifically allowed is forbidden.

KEEP THE FIREWALL EFFICIENT

- **Sort** traffic types according workload
- Write rules (rejecting then acceptance) according to descending workload

DEFAULT FILTERING

- Accept or reject according to first concordance
- specific rules checked before general rules

DEFAULT POLICY

Anything that is not specifically allowed is forbidden.

KEEP THE FIREWALL EFFICIENT

- **Sort** traffic types according workload
- Write rules (rejecting then acceptance) according to descending workload

DEFAULT FILTERING

- Accept or reject according to first concordance
- specific rules checked before general rules

DEFAULT POLICY

Anything that is not specifically allowed is forbidden.

KEEP THE FIREWALL EFFICIENT

- **Sort** traffic types according workload
- Write rules (rejecting then acceptance) according to descending workload

REJECT OR DENY A PACKET?

- ❶ **Reject** $\Rightarrow$ returns an ICMP message
- ❷ **Deny** $\Rightarrow$ no message

DENY IS SAFER SINCE EACH RESPONSE

- duplicates traffic $\rightarrow$ maybe on purpose
- can be used for DOS attack
- contains some information useful for an attacker

REJECT OR DENY A PACKET?

- ❶ **Reject** $\Rightarrow$ returns an ICMP message
- ❷ **Deny** $\Rightarrow$ no message

DENY IS SAFER SINCE EACH RESPONSE

- duplicates traffic $\rightarrow$ maybe on purpose
- can be used for DOS attack
- contains some information useful for an attacker

SAMPLE RULES

- Local network: data server, mail server, DNS server
- DMZ: Web server, application server
- Outside network: mail server, remote DNS server

RULES FOR SINGLE FIREWALL

rules	IP source	IP dest.	Service	Action
1	All	Web server	HTTP	Accept
2	All	Mail server	SMTP	Accept
3	Mail server	All	SMTP	Accept
4	Local network	All	HTTP, HTTPS, FTP, SSH	Accept
5	Local DNS	All	DNS	Accept
6	All	All	All	Deny

RULES FOR PAIR OF FIREWALLS

- First firewall: see single firewall rules
- Second firewall:

Rules	source IP	dest. IP	Service	Action
1	Local mail server	Mail server	SMTP	Accept
2	Local network	All	HTTP,HTTPS, FTP, SSH	Accept
5	Local DNS	All	DNS	Accept
4	All	All	All	Deny

OUTLINES

2 ENTERING PACKET FILTERING

FILTER REMOTE SOURCE @

Sender identified with packet source @ → beware **spoofing**

FORBIDDEN PACKETS

- IP @ from local network
- Private IP @ from classes A, B et C, multicast (D), reserved (E)
- loopback @
- some broadcast @ (eg. 0.0.0.0 for DHCP)

SELECT ONLY SOURCE @ COMING FROM

- remote servers (HTTP, FTP)
- remote clients connected with local servers

FILTER REMOTE SOURCE @

Sender identified with packet source @ → beware **spoofing**

FORBIDDEN PACKETS

- IP @ from local network
- Private IP @ from classes A, B et C, multicast (D), reserved (E)
- loopback @
- some broadcast @ (eg. 0.0.0.0 for DHCP)

SELECT ONLY SOURCE @ COMING FROM

- remote servers (HTTP, FTP)
- remote clients connected with local servers

FILTER REMOTE SOURCE @

Sender identified with packet source @ → beware **spoofing**

FORBIDDEN PACKETS

- IP @ from local network
- Private IP @ from classes A, B et C, multicast (D), reserved (E)
- loopback @
- some broadcast @ (eg. 0.0.0.0 for DHCP)

SELECT ONLY SOURCE @ COMING FROM

- remote servers (HTTP, FTP)
- remote clients connected with local servers

FILTER LOCAL DEST. @

- The network interface disregards packets for which it is not the destination
- Except for broadcast packets

FILTERING REMOTE SOURCE PORTS

THE PORT SOURCE IDENTIFIES THE REMOTE APPLICATION

- Requests from remote clients to local servers → same model
- ⇒ source ports without any privilege for clients (1024 to 65535)
- Responses from remote servers to local clients → different model
- ⇒ Specific source port for each service

FILTERING REMOTE SOURCE PORTS

THE PORT SOURCE IDENTIFIES THE REMOTE APPLICATION

- Requests from remote clients to local servers → same model
⇒ source ports without any privilege for clients (1024 to 65535)
- Responses from remote servers to local clients → different model
⇒ Specific source port for each service

FILTERING LOCAL DESTINATION PORT

- Specific number for requests from remote clients to local services
- Numbers > 1024 for responses from remote servers to local clients

FILTERING STATE OF TCP CONNEXION

SAME STATE SETS FOR ALL TCP CONNEXIONS

- Client: *SYN* for first packet, then *ACK* for all others
- Server: always *ACK*

FILTERING STATE OF TCP CONNEXION

SAME STATE SETS FOR ALL TCP CONNEXIONS

- Client: *SYN* for first packet, then *ACK* for all others
- Server: always *ACK*

PROBES AND SCANS

PROBE = connexion attempt or ask for port response

SCAN = several probes

LOG ANALYSIS TO SET AN ALERT THRESHOLD

- Single attempts: ignore them
- Otherwise: strengthen rules, keep logs, warn local users

PROBES AND SCANS

PROBE = connexion attempt or ask for port response

SCAN = several probes

LOG ANALYSIS TO SET AN ALERT THRESHOLD

- Single attempts: ignore them
- Otherwise: strenghten rules, keep logs, warn local users

DENY OF SERVICE

NO PERFECT PREVENTION

TCP AND SYN FLOODING : system ressource are used until connexions are blocked

⇒ filter spoofed @ and recompile kernel

PING FLOODING ⇒ disable or limit entering pings

UDP FLOODING (stateless protocol) → the system answers to UDP probes instead of valid traffic

⇒ open only ports that are strictly necessary

ICMP-REDIRECT BOMBS : the hacker modifies your routing tables to let the traffic go through systems she controls

DISTRIBUTED DoS : simultaneous attacks from several locations

DENY OF SERVICE

NO PERFECT PREVENTION

TCP AND SYN FLOODING : system ressource are used until connexions are blocked

⇒ filter spoofed @ and recompile kernel

PING FLOODING ⇒ disable or limit entering pings

UDP FLOODING (stateless protocol) → the system answers to UDP probes instead of valid traffic

⇒ open only ports that are strictly necessary

ICMP-REDIRECT BOMBS : the hacker modifies your routing tables to let the traffic go through systems she controls

DISTRIBUTED DoS : simultaneous attacks from several locations

DENY OF SERVICE

NO PERFECT PREVENTION

TCP AND SYN FLOODING : system ressource are used until connexions are blocked

⇒ filter spoofed @ and recompile kernel

PING FLOODING ⇒ disable or limit entering pings

UDP FLOODING (stateless protocol) → the system answers to UDP probes instead of valid traffic

⇒ open only ports that are strictly necessary

ICMP-REDIRECT BOMBS : the hacker modifies your routing tables to let the traffic go through systems she controls

DISTRIBUTED DoS : simultaneous attacks from several locations

DENY OF SERVICE

NO PERFECT PREVENTION

TCP AND SYN FLOODING : system resource are used until connexions are blocked

⇒ filter spoofed @ and recompile kernel

PING FLOODING ⇒ disable or limit entering pings

UDP FLOODING (stateless protocol) → the system answers to UDP probes instead of valid traffic

⇒ open only ports that are strictly necessary

ICMP-REDIRECT BOMBS : the hacker modifies your routing tables to let the traffic go through systems she controls

DISTRIBUTED DoS : simultaneous attacks from several locations

DENY OF SERVICE

NO PERFECT PREVENTION

TCP AND SYN FLOODING : system ressource are used until connexions are blocked

⇒ filter spoofed @ and recompile kernel

PING FLOODING ⇒ disable or limit entering pings

UDP FLOODING (stateless protocol) → the system answers to UDP probes instead of valid traffic

⇒ open only ports that are strictly necessary

ICMP-REDIRECT BOMBS : the hacker modifies your routing tables to let the traffic go through systems she controls

DISTRIBUTED DoS : simultaneous attacks from several locations

OTHER ATTACKS

- filesystem overloading

⇒ ressource limitation, dedicated partitions

- servers blocked due to too bulky or erroneous data

⇒ keep an eye on updates

- overworked CPU

⇒ limit services and deny packets

OTHER ATTACKS

- filesystem overloading

⇒ ressource limitation, dedicated partitions

- servers blocked due to too bulky or erroneous data

⇒ keep an eye on updates

- overworked CPU

⇒ limit services and deny packets

OTHER ATTACKS

- filesystem overloading

⇒ ressource limitation, dedicated partitions

- servers blocked due to too bulky or erroneous data

⇒ keep an eye on updates

- overworked CPU

⇒ limit services and deny packets

ABOUT FILTERING

SOURCE ROUTING

force routers to send packets along a specific path ⇒ **to disable!**

PACKET FRAGMENTATION

Only the first fragment contains source and destination ports

→ if *IP masquerading* is on, fragments have to be regrouped before filtering

⇒ otherwise, extra fragments could be inserted (fragment overlap or tiny fragment attacks)

ABOUT FILTERING

SOURCE ROUTING

force routers to send packets along a specific path $\Rightarrow$ **to disable!**

PACKET FRAGMENTATION

Only the first fragment contains source and destination ports

- $\rightarrow$ if *IP masquerading* is on, fragments have to be regrouped before filtering
- $\Rightarrow$ otherwise, extra fragments could be inserted (**fragment overlap** or **tiny fragment attacks**)

OUTLINES

3 FILTERING EXITING PACKETS

WHY?

- protect against local errors
 - check if packets from local applications are not sent outside
 - block attacks coming from your network

WHY?

- protect against local errors
- check if packets from local applications are not sent outside
- block attacks coming from your network

WHY?

- protect against local errors
- check if packets from local applications are not sent outside
- block attacks coming from your network

FILTER LOCAL SOURCE @

- small site: @ is always the same for a given computer
- DHCP → transactions between clients and servers
⇒ control exiting packets
- firewall → packet source @ = firewall @ (Network Address Translation)

FILTER LOCAL SOURCE @

- small site: @ is always the same for a given computer
 - DHCP → transactions between clients and servers
- ⇒ control exiting packets
- firewall → packet source @ = firewall @ (Network Address Translation)

FILTER LOCAL SOURCE @

- small site: @ is always the same for a given computer
- DHCP → transactions between clients and servers
⇒ control exiting packets
- firewall → packet source @ = firewall @ (Network Address Translation)

FILTER REMOTE DEST. @

- to send exiting packets to specific targets
- useful when
 - dest @ = specific servers (DHCP)
 - packets targeted to remote clients (SSH)

FILTER REMOTE DEST. @

- to send exiting packets to specific targets
- useful when
 - dest @ = specific servers (DHCP)
 - packets targeted to remote clients (SSH)

FILTERING LOCAL SOURCE PORT

FROM LOCAL CLIENTS

- port > 1024
- rules depend on each considered port and protocol

FROM LOCAL SERVER

- Always coming from the same port
- limit server access to their port on the firewall
- ⇒ any service processed from the outside is protected
- ⇒ remote sites are not hit by local traffic

FILTERING LOCAL SOURCE PORT

FROM LOCAL CLIENTS

- port > 1024
- rules depend on each considered port and protocol

FROM LOCAL SERVER

- Always coming from the same port
- limit server access to their port on the firewall
- ⇒ any service processed from the outside is protected
- ⇒ remote sites are not hit by local traffic

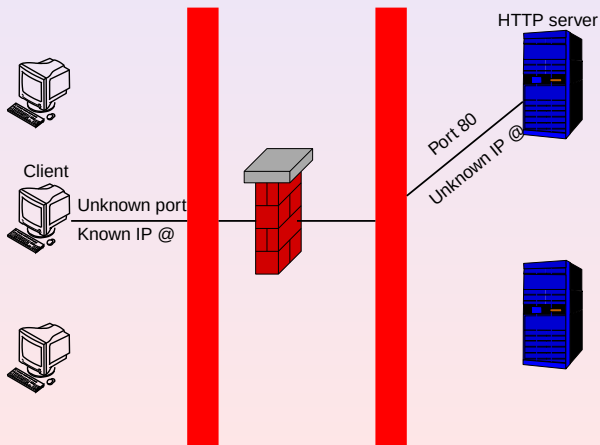
FILTER LOCAL DEST. PORT

LOCAL CLIENTS: CONNECTED TO A SPECIFIC PORT

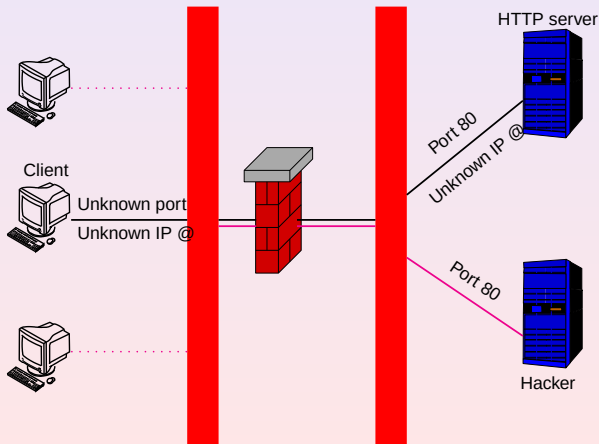
- force those clients to connect to this port
- ⇒ block errors and port probes coming from local network

Figures

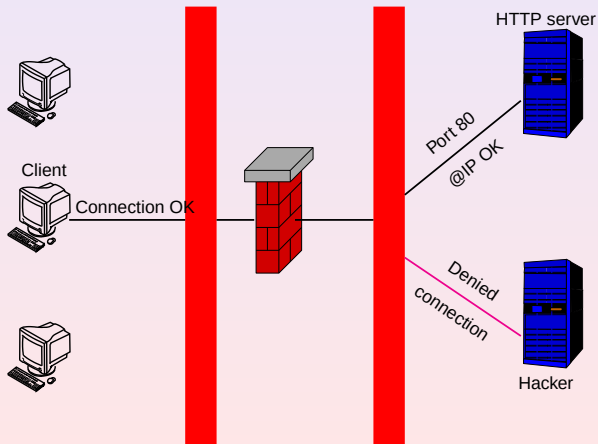
FILTERING LIMITS (EX. HTTP) 1/2



LIMITE DU FILTRAGE (EX. HTTP) 2/2

[◀ Back](#)

STATEFUL FIREWALL - TCP

[◀ Back](#)

INPUT AND OUTPUT CHAINS

[◀ Back](#)